Corrective, preventive and improvement actions: FaceAttend EDU

Jonattan Steven Rizo Solano

Diego Andrés Gutiérrez Núñez

Juan David Arboleda Perdomo

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje

Technologist in Software Analysis and Development, Cohort 3145556

November 4, 2026

Abstract

This document defines the documentation framework for managing corrective, preventive, and improvement actions in the software lifecycle. It establishes principles, roles, workflow, risk criteria, evidence, indicators, and closure mechanisms based on continuous improvement. Its purpose is to guarantee traceability, verifiable effectiveness, and alignment with quality management standards, systems engineering, and professional documentation presentation criteria.

Keywords: corrective actions, preventive actions, continuous improvement, software quality

Contents

Abstract 2
Introduction 4
- Objective 4
- Scope 5
- Definitions 5

Regulatory and Conceptual Framework 6
- Quality Management 6
- Software Lifecycle 6
- Risks and Opportunities 7
- Product and In-Use Quality 7
- Documentation Presentation 7

Documentation and Action Management Process 9
- Identification and Registration 9
- Triage and Classification 10
- Root Cause Analysis 10
- Impact and Risk Evaluation 12
- Action Planning 12
- Controlled Implementation 13
- Implementation Verification 14
- Efficacy Validation 14
- Closure and Feedback 15

Corrective, preventive and improvement actions: FaceAttend EDU

Introduction

Software quality management cannot be limited to detecting defects, incidents, or audit findings. It requires a structured mechanism to transform those events into verifiable, traceable, and effective actions. In this context, corrective, preventive, and improvement actions constitute a technical and organizational feedback cycle that allows reducing variability, improving product reliability, and strengthening the maturity of development processes.

This document is based on quality management systems standards (International Organization for Standardization [ISO], 2015), system and software lifecycle processes (International Organization for Standardization & International Electrotechnical Commission [ISO & IEC], 2017), risk management (ISO, 2018), and system and software quality models (ISO & IEC, 2011). Its documentation structure also seeks to align with internationally recognized professional and academic presentation criteria, including the principles of clarity, heading hierarchy, author-date citation, and reference listing characteristic of the seventh edition of APA Standards (American Psychological Association [APA], 2020).

From a software engineering perspective, a corrective action should not be confused with a superficial fix. A fix eliminates the visible effect of a problem, while a corrective action attacks the root cause to prevent recurrence. Similarly, a preventive action does not wait for the failure to materialize, but acts on identified risks, weak trends, near-misses, or controlled strengthening opportunities. An improvement action, in turn, does not necessarily respond to a nonconformity, but to the deliberate intention to increase efficacy, efficiency, maintainability, security, or user experience.

Objective

Establish the documentation framework, operational criteria, and responsibilities associated with the identification, registration, analysis, implementation, verification, validation, and closure of corrective, preventive, and improvement actions applicable to the software lifecycle.

Scope

This document applies to all software products developed, maintained, integrated, or operated by the organization, including web applications, backend services, internal libraries, embedded components, continuous integration and deployment pipelines, infrastructure as code, technical documentation, quality assurance processes, and post-deployment support activities.

It also applies to findings from internal audits, code reviews, functional and non-functional testing, production incidents, user complaints, service monitoring, technical debt analysis, security assessments, architecture reviews, and improvement opportunities identified by multidisciplinary teams.

It does not replace specific procedures for security incident response, operational continuity, or personal data management; however, it must integrate with them when an event requires root cause analysis and systemic actions.

Definitions

For the purposes of this document, the following operational definitions are adopted:

- **Corrective action**: action taken to eliminate the root cause of a detected nonconformity and prevent its recurrence.
- **Preventive action**: action taken to eliminate the cause of a potential nonconformity, in order to prevent its occurrence.
- **Improvement action**: action aimed at increasing the efficacy, efficiency, quality, security, maintainability, or perceived value of the product or process, without there necessarily being an active nonconformity.
- **Nonconformity**: failure to meet a specified requirement, whether functional, regulatory, process, security, performance, or quality.
- **Risk**: effect of uncertainty on project, product, or service objectives, expressed in terms of probability and impact.
- **Opportunity**: risk with a positive effect, susceptible to being leveraged to improve performance, reduce cost, increase reliability, or strengthen user experience.

- **Root cause**: fundamental factor whose elimination prevents the problem from recurring. It should not be confused with the symptom, the triggering event, or individual responsibility.
- **Efficacy**: degree to which the implemented action achieves the intended result, measured through objective evidence and previously defined criteria.
- **Verification**: confirmation that the action was implemented according to the approved plan.
- **Validation**: confirmation that the implemented action produces the desired effect in the real context of use.
- **Record**: document, data, or evidence that preserves the history of the process and allows reconstructing decisions, responsible parties, dates, and results.

Regulatory and Conceptual Framework

Quality Management

ISO 9001:2015 establishes that the organization must determine and act on nonconformities, evaluate the need to act to eliminate causes and prevent recurrence, and maintain documented information as evidence of the nature of nonconformities and the results of actions taken (ISO, 2015, clause 10.2). It also promotes continuous performance improvement through data analysis, risk and opportunity evaluation, and management review (ISO, 2015, clause 10.3).

A relevant point for organizations that migrated from earlier versions of ISO 9001 is that the 2015 edition no longer requires an independent clause called "preventive action." Instead, it incorporates risk-based thinking as a transversal principle. However, in software engineering, it maintains full operational validity to distinguish between reactive, proactive, and improvement actions, because the types of evidence, timing, and efficacy criteria differ significantly.

Software Lifecycle

ISO/IEC/IEEE 12207:2017 describes processes for the definition, implementation, maintenance, and improvement of system and software lifecycles (ISO & IEC, 2017). From this

perspective, corrective, preventive, and improvement actions do not belong exclusively to the testing or operation phase, but must be integrated into requirements, design, implementation, verification, validation, deployment, support, and retirement.

In agile, DevOps, or DevSecOps environments, this integration implies that each action must be linked to living artifacts: backlog, user stories, technical tasks, pull requests, pipelines, service metrics, observability dashboards, and documentation repositories. Traceability should not depend solely on static documents, but on verifiable links between technical evidence and business decisions.

Risks and Opportunities

The management of preventive and improvement actions must articulate with ISO 31000:2018, which provides principles and guidelines for integrating risk management into the governance, planning, operations, and reporting of the organization (ISO, 2018). In software, this means treating technical, operational, security, compliance, reputation, and continuity risks as legitimate inputs to trigger actions before they materialize as incidents.

Product and In-Use Quality

The ISO/IEC 25010:2011 model allows classifying quality characteristics as functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability, as well as quality in use (ISO & IEC, 2011). The actions documented in this framework must be able to map to these characteristics, so that the organization not only closes findings but demonstrates measurable improvement in relevant product attributes.

Documentation Presentation

Table 1

Documentation Presentation

Action Type	Origin / Detected Finding	Central Purpose	Minimum Evidence and Verifiable Action
Improvement	Slow progress of the web and mobile application relative to the project plan.	Make substantial progress in upcoming reviews, prioritizing critical functionalities, missing screens, and minimum viable product construction.	Prioritized backlog, sprint planning, sprint review, progress metrics, release notes, screenshots of newly implemented screens, and deployment or build evidence.
Improvement	Differences between the database, the developed frontend, existing mockups, and the applied visual identity, including solid colors without gradient and screens not represented in design.	Correct the inconsistency between data model, graphical interface, and mockups, improving visual coherence, design traceability, and correct implementation of the web and mobile application.	Database/frontend/mockup comparison, updated style guide, versioned design files in Figma or equivalent tool, visual adjustment pull requests, UX/UI review evidence, and screen traceability document.
Corrective	Deficiency in the "Forgot password" flow, including views, validations, navigation, or inadequate user experience.	Correct the password recovery process to ensure a clear, secure, functional, and consistent flow between web and mobile application.	Defect ticket, video or screenshot of the error, functional and security test cases, associated pull request, regression tests, deployment evidence, and flow abandonment or success metric before/after.
Corrective	Date fields do not allow correct selection or present error in the mobile application.	Restore date selection functionality on mobile devices, ensuring iOS/Android compatibility, adequate validation, and correct user experience.	Mobile bug report, screenshot or video of the error, corrected commit or pull request, manual or automated tests on emulators/real devices, mobile build evidence, and verified closure record.
Corrective	Navigation problems in the mobile application, such as confusing routes, non-functional buttons, lack of return, or interrupted flows.	Correct mobile navigation to ensure consistent, accessible, and unblocked movement between main and secondary screens.	Updated navigation map, navigation defect list, corrected pull request, usability test cases, mobile device testing evidence, deployment record, and verification minutes.
Preventive	Need to add security questions section in the account, recovery, or identity verification module.	Implement an additional authentication control that reduces the risk of unauthorized access, identity impersonation, or improper account recovery.	Approved security requirement, interface and API design, updated risk matrix, secure storage of hashed or encrypted responses, attempt limits, audit logs, test cases, and security review evidence.
Preventive	Need to review security protocols to prevent forgery, impersonation, request manipulation, or improper access.	Define and implement technical and procedural controls that prevent identity, token, request, session, or sensitive information forgery.	Threat model or threat analysis, OWASP ASVS checklist or similar, HTTPS/HSTS configuration, signed and expiring tokens, CSRF protection, rate limiting, input validation, security tests, review report, and mitigation evidence.

Action Type	Origin / Detected Finding	Central Purpose	Minimum Evidence and Verifiable Action
Corrective	Only the administrator role is displayed; other roles defined for the system are missing.	Implement and verify correct role and permission management according to the approved RBAC matrix, ensuring each profile accesses only its permitted functionalities.	Updated RBAC matrix, role seed data or migrations, implemented pull request, role-based authorization tests, interface screenshots per profile, deployment evidence, and permission validation record.
Corrective	The mobile mockup does not match the actual development progress.	Restore traceability between approved design and mobile implementation, correcting either the development, the mockup, or both, according to the approved technical and functional decision.	Mockup/mobile application comparison, decision minutes on design baseline, updated Figma or mockup, corrected backlog, alignment pull request, UX/UI review evidence, and closure record.

Documentation and Action Management Process

The recommended process consists of nine sequential stages, with feedback in case of inefficacy. Each stage must generate objective and traceable evidence.

Identification and Registration

Every action must begin with a unique record. The identifier must be persistent, human-readable, and preferably correlatable with incident management systems, repository, pipeline, or quality tools.

Minimum record fields:

- Unique action identifier.
- Detection date.
- Registration date.
- Requester or detector.
- Affected product, service, module, or component.
- Affected version or release.
- Environment: development, testing, pre-production, production.
- Objective description of the observed fact.
- Estimated initial impact.

- Perceived urgency.
- Preliminary classification: corrective, preventive, or improvement.
- Evidence links: logs, screenshots, test reports, tickets, audits, metrics.
- Assigned responsible party.
- Initial status: open.

The description must be factual. Accusatory language, unverified assumptions, or premature conclusions about blame should be avoided. The focus is on the system, the process, and the evidence, not on people.

Triage and Classification

Triage determines whether the event requires a formal action or can be managed as an ordinary task. Criteria for escalating to a formal action include:

- Impact on external users.
- Security or privacy risk.
- Regulatory or contractual non-compliance.
- Impact on availability, integrity, or confidentiality.
- Historical recurrence.
- High cost of deferred correction.
- Late detection relative to the optimal prevention phase.
- Need for change in architecture, process, or governance.

During triage, the provisional classification is confirmed and priority is assigned. If the same event contains multiple causes or effects, it can be broken down into several linked actions, maintaining parent-child traceability.

Root Cause Analysis

Root cause analysis is mandatory for corrective actions and recommended for high-impact preventive actions. Describing the symptom is not enough. The systemic factor that allowed the occurrence or lack of early detection must be identified.

Admissible methods:

- Five whys.
- Ishikawa or fishbone diagram.
- Barrier analysis.
- Fault tree analysis.
- Change analysis.
- Timeline review.
- Structured root cause analysis for major incidents.

Minimum dimensions to examine in software:

- Ambiguous, incomplete, or changing requirements.
- Insufficient architectural design.
- Defective implementation.
- Inadequate test coverage.
- Non-representative test data.
- Inconsistent environment configuration.
- Vulnerable dependency management.
- Deficient change control.
- Limited observability.
- Human factors: fatigue, cognitive load, communication, training.
- Organizational processes: deadline pressure, lack of technical review, misaligned incentives.

The result must include:

- Root cause statement.
- Secondary contributors.
- Reason why the problem was not detected earlier.
- Potential scope of impact.
- Confidence level of the analysis.
- Evidence supporting each conclusion.

If a single root cause is not identified, a set of contributing causes must be documented and those with the greatest controllable influence prioritized. If the cause remains unknown after reasonable effort, compensatory controls must be implemented, enhanced monitoring defined, and the analysis reopened if recurrence occurs.

Impact and Risk Evaluation

Before approving the plan, residual risk must be evaluated. This evaluation considers probability, impact, detectability, and propagation speed.

Evaluation criteria:

- Number of affected or potentially affected users.
- Service criticality.
- Exposure of personal or sensitive data.
- Financial impact.
- Reputational impact.
- Regulatory non-compliance.
- Affected external dependencies.
- Exposure time window.
- Recovery capacity.
- Cost and time of action implementation.

The evaluation must allow deciding whether immediate containment, scheduled correction, architectural change, documentation update, or committee review is required.

Action Planning

The plan must be specific, measurable, achievable, relevant, and time-bound. Each action must have a single responsible party, commitment date, acceptance criteria, and expected evidence.

Plan components:

- Containment actions, if applicable.
- Actions to correct the immediate effect.

- Actions on root cause.
- Associated preventive actions.
- Derived improvement actions.
- Technical documentation update.
- Test case update.
- Monitoring and alert update.
- Training, if the process changes.
- Communication to stakeholders.
- Rollback plan.
- Effort and resource estimation.
- Technical or organizational dependencies.
- Success criteria.
- Efficacy validation period.

Actions should avoid generalities such as "improve testing" or "train the team." Proper formulation: "Add regression test suite for the payment flow with timeout scenario coverage, idempotent retries, and accounting consistency validation, before production deployment."

Controlled Implementation

Implementation must respect the organization's software engineering controls:

- Branches or commits linked to the action identifier.
- Peer code review.
- Continuous integration pipeline execution.
- Unit, integration, system, and, where applicable, acceptance testing.
- Static and security analysis.
- Artifact and version management.
- Deployment with feature flags, canary release, or blue-green, depending on risk.
- Change recording in release notes.
- Update of diagrams, APIs, data models, and runbooks.

- Preservation of approval and execution evidence.

In regulated or critical environments, implementation may require prior approval from a change committee. In agile environments, it can be done through pull request with designated reviewers and verifiable automation. The essential thing is that the action is not lost in the backlog without traceability.

Implementation Verification

Verification answers the question: was what was planned done?

Typical evidence:

- Link to commit, merge request, or pull request.
- Pipeline result.
- Test report.
- Deployment record.
- Screenshot of modified configuration.
- Updated document.
- Training minutes.
- Policy or procedure change.

If verification fails, the action does not advance to validation. The plan or execution must be corrected, as applicable.

Efficacy Validation

Validation answers the question: did the action produce the expected effect?

For corrective actions, efficacy is confirmed when:

- The same failure mode does not recur within the defined period.
- Affected indicators return to acceptable thresholds.
- Specific tests pass consistently.
- Implemented controls are observable and sustainable.

For preventive actions, efficacy is confirmed when:

- The treated risk decreases to an accepted level.

- No associated incidents materialize during the monitoring period.
- Preventive controls execute automatically or according to defined frequency.

For improvement actions, efficacy is confirmed when:

- The target metric is achieved.
- Other quality characteristics are not degraded.
- The benefit is operationally and economically sustainable.

The validation period must be defined before closure. It can be 30, 60, 90, or 180 days, depending on criticality, occurrence frequency, and nature of the change. If the action is not effective, it must be reopened, root cause analysis expanded, or a new intervention designed.

Closure and Feedback

Closure requires approval from the quality responsible party or equivalent role. The closed file must contain:

- Original record.
- Root cause analysis.
- Risk evaluation.
- Approved plan.
- Implementation evidence.
- Verification.
- Efficacy validation.
- Lessons learned.
- Derived updates to processes, standards, risks, tests, and training.
- Closure date.
- Signature or electronic record of the responsible party.

Closure is not an administrative end. It must feed the organizational knowledge system: defect databases, review checklists, static analysis rules, test cases, runbooks, risk matrices, and training programs.

In small organizations, the same role may assume multiple responsibilities, provided minimum segregation is preserved between who implements and who validates efficacy, especially in critical changes.